Data analysis and bio-statistics

Izan Flores Tortosa & Jordi Vivas Sakuta

Exercises for practice 5

All the exercises for this practice will be written in a single R markdown script file (.Rmd extension). Don't forget to use comments to indicate each exercise in the code and what you are trying to achieve within each section. Please, keep each exercise statement in your R script as a comment in order to make the code more readable.

The exercises in practice 5 will be carried out using the provided dataset in a CSV file. This dataset contains the measurements of a real experiment which evaluates the effect of two drugs: fenofibrate and niacin in type 2 diabetes patients. For a complete description of the experiment please refer to the paper: https://doi.org/10.1016/j.atherosclerosis.2014.12.006

The first step before starting the exercise consist in loading the dataset using the read.csv2() function. Here, we are not going to use the metadata columns. So, it is strongly recommended to use the following code to load your data and discard the unused columns (variables) before starting the exercises.

```
#Edit this code chunk to successfully load the CSV data
df <- read.csv2("bdLOWHDL_metabolomica.csv", header = T, sep = ";", dec = ",")
df <- df[,(27:ncol(df))]
```

1- A function to calculate the correlation matrix

Write an R function with the following prototype:

correlationMat(x)

where x is a matrix or data.frame containing the variables and observations to calculate the correlation matrix

return value: the correlation matrix for all variables in the dataset.

The correlationMat() function must calculate the correlation matrix for all the variables in x. Here there are some tips to calculate the correlation matrix:

- Start by creating an empty square matrix to store all correlation results. You can use the following code:

```
corMat <- matrix( nrow = ncol(x), ncol = ncol(x) )
```

- Add names for rows and columns of corMat to keep track of variables. You can use the following code:

```
colnames(corMat) <- colnames(x)
rownames(corMat) <- colnames(x)
```

- Use two nested for loops to traverse all cells of the corMat.
- Calculate the correlation of each variable pair using the **cor()** function.
- Be aware of possible NA values in each variable pair. You can use the following code to filter out NA values before computing the correlations:

```
    na_v1 <- which(is.na(v1)) #get na positions of the first variable
    na_v2 <- which(is.na(v2)) #get na positions of the second variable
    na_index <- unique(c(na_v1, na_v2)) #combine na positions of both variables
```

```
#Write your code here
correlationMat<-function(df){
corMat <- matrix(nrow = ncol(df), ncol = ncol(df))
colnames(corMat) <- colnames(df)
rownames(corMat) <- colnames(df)
v1<-df[,1]
v2<-df[,2]


for(i in 1:ncol(corMat)){
  for(j in 1:ncol(corMat)){
    v1<-df[,j]
    v2<-df[,i]
    na_v1 <- which(is.na(v1)) #get na positions of the first variable
    na_v2 <- which(is.na(v2)) #get na positions of the second variable
    na_index <- unique(c(na_v1, na_v2)) #combine na positions of both variables
    v1<-v1[-na_index]
    v2<-v2[-na_index]
    corMat[i,j]<-cor(v1,v2)
  }
}
return( corMat)
}
```

2-Testing the correlationMat() function

Use your **correlationMat()** function to calculate the correlation matrix of df data. Store the result in a variable. Then, use the following code to calculate the same correlation matrix using built-in R functions:

```
rCorMat <- cor(df, use ="pairwise.complete.obs")
```

Finally compare your results with R results. To do so, you can manually inspect the two matrices, however it is much faster to use the following code to let R compare the two matrices for you:

```
identical(round(myCorMat, digits = 2), round(rCorMat, digits = 2))
```

If this code returns TRUE then you correlation matrix is identical to the R calculated matrix.

```
#Write your code here
rCorMat <- cor(df, use ="pairwise.complete.obs")

myCorMat<-correlationMat(df)
identical(round(myCorMat, digits = 2), round(rCorMat, digits = 2))
```

```
## [1] TRUE
```

3-Select a pair of highly correlated variables

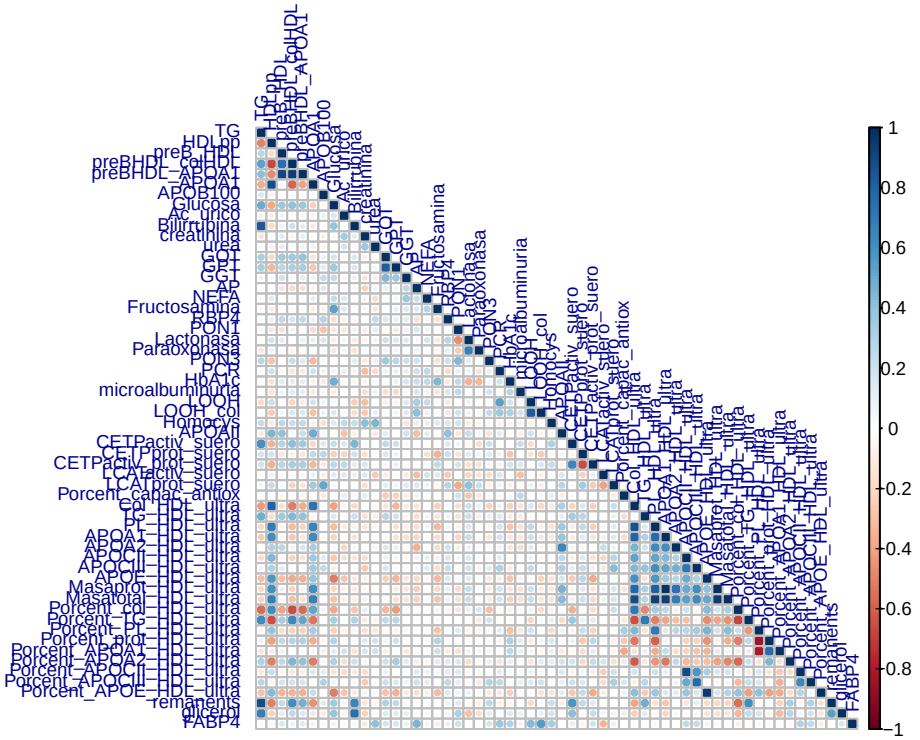
Install the **corrplot** package and load it using *library(corrplot)*. Then, display the correlation matrix with the following code:

```
install.packages("corrplot")
```

```
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
corrplot(myCorMat, method="circle", type="lower", tl.cex = 0.6,
         tl.col = "darkblue", cl.pos = "r", cl.cex = 0.6, cl.ratio = 0.08)
```



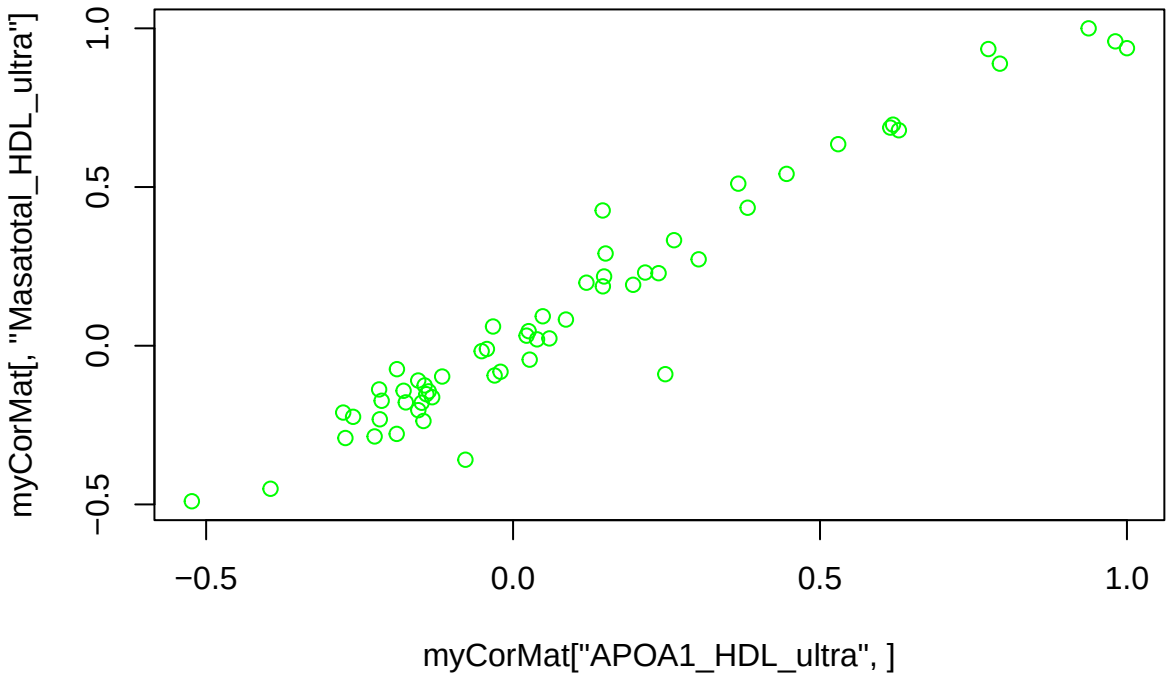
Looking at the correlation matrix plot, select a pair of highly correlated variables. APOA1_HDL_ultra vs Masatotal_HDL_ultra APOA1_HDL_ultra vs masaprot_hdl_ULTRA

4-Variable relationship and linear modelling

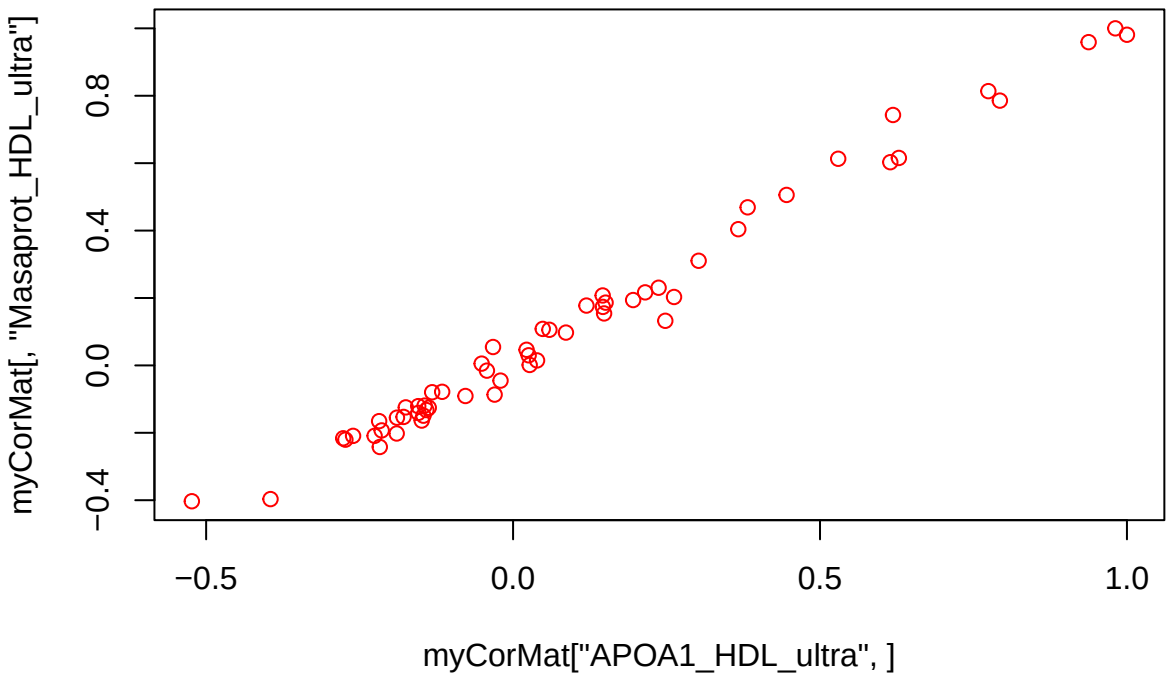
4.1- Create a scatter plot of the two selected variables.

```
#Write your code here

plot(myCorMat["APOA1_HDL_ultra",],myCorMat[,"Masatotal_HDL_ultra"],col="green")
```




```
plot(myCorMat["APOA1_HDL_ultra",],myCorMat[,"Masaprot_HDL_ultra"],col="red")
```



4.2 - Execute the following code to create a vector which contains the row numbers of just 90% of randomly selected observation in the df dataset. This will allow us to train our linear model with a subset of the data (90%) and use the remaining data (10%) to evaluate the model.

```
randomObs <- sort(sample( 1:nrow(df), round(0.9*nrow(df))))
```

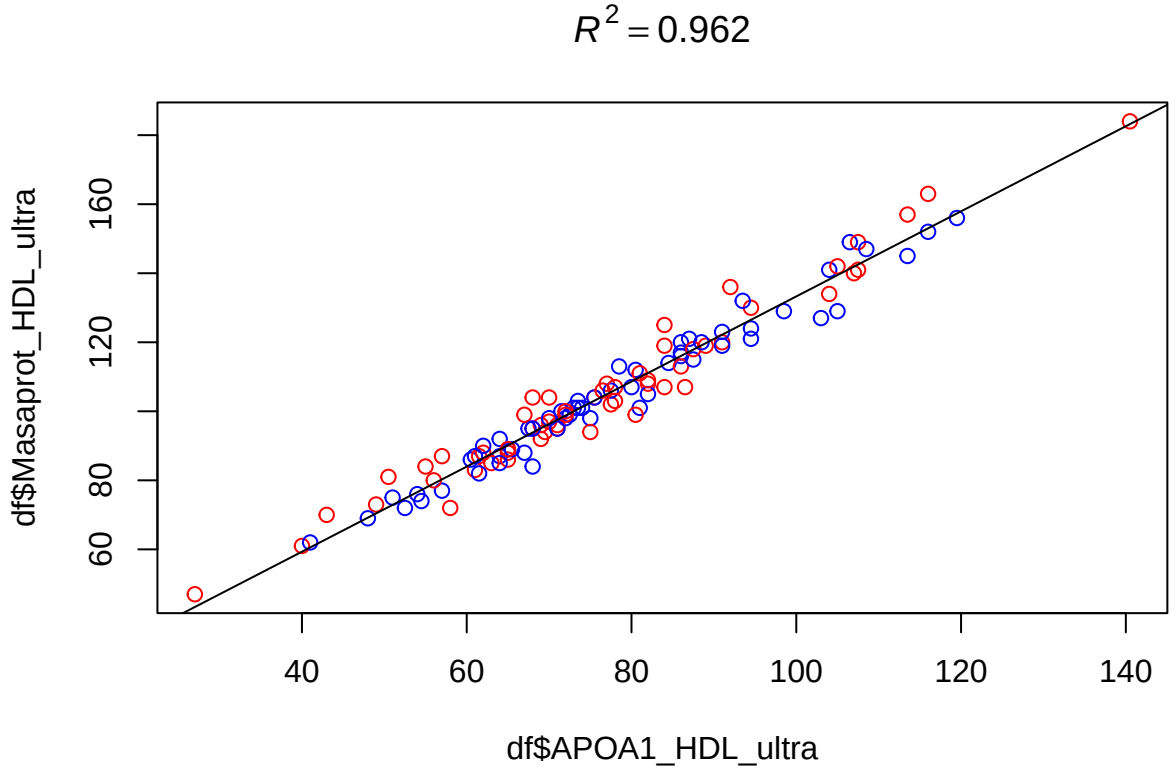
4.3 - Use the **lm()** R function to create a linear model using only the randomObs selected observations. Use the **abline()** R function to display the linear regression produced by lm overlaid on the scatter plot. Display also the R^2 value in the plot, you can use the following plot main title:

```
main = bquote(italic(R)^2 == .(format(summary(myLm)$r.squared, digits = 3)))
```

```
#Write your code here


myLm<-lm( Masaprot_HDL_ultra~ APOA1_HDL_ultra,df[randomObs,])
colors<-c("APOA1_HDL_ultra"="red", "Masaprot_HDL_ultra"="blue")
plot(df$APOA1_HDL_ultra,df$Masaprot_HDL_ultra,col=colors,main =
       bquote(italic(R)^2 == .(format(summary(myLm)$r.squared, digits = 3))))

abline(myLm)
```



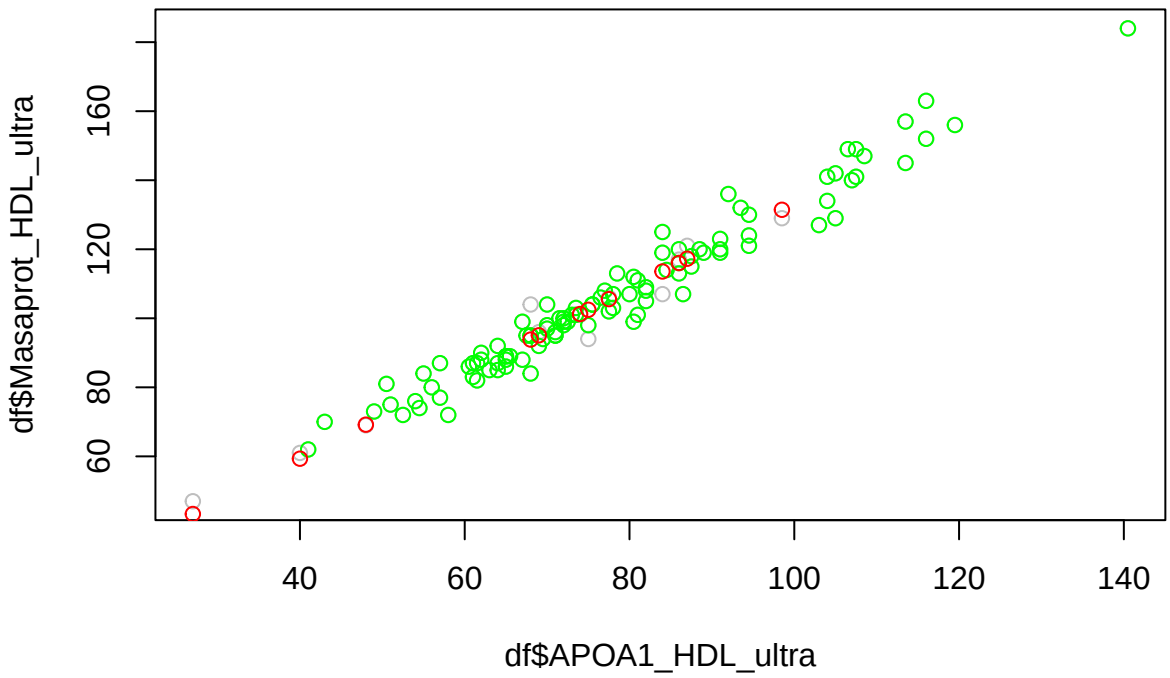
4.4 - Use the **predict()** R function to use your liner model to predict the values of your second variable using the remaining 10% observations of your first variable. In order to see the performance of the prediction, create a scatter plot with the following layers:

- A scatter plot of your two selected variables with points represented in grey color (all observations).
- A scatter plot of the 90 % of the data used to create the linear model. Display the points in green.
- A scatter plot of the 10 % of the data used to evaluate the model and the prediction results in red.

```
#Write your code here



plot(df$APOA1_HDL_ultra,df$Masaprot_HDL_ultra,col="grey")
points(df$APOA1_HDL_ultra[randomObs],df$Masaprot_HDL_ultra[randomObs],col="green")
prediccio<- predict(myLm,df[-randomObs,])
points(df$APOA1_HDL_ultra[-randomObs],prediccio,col="red")
```


df$Masaprot_HDL_ultra
60
80
120
160
40
60
80
100
120
140
df$APOA1_HDL_ultra